

The DevOps Engine

What the engine does — and what you, the miner, should check once your GPU miner is started.

This handbook explains the operations engine behind every InfraNex miner: how it validates a GPU host with a ten-step inspection pipeline, installs a subnet's real requirements from its official repository, launches the workload behind explicit approval gates, and keeps a ninety-second watch on hardware, telemetry and chain drift. It closes with the checks that matter most — the first thirty minutes after launch, and the steady-state rhythm that keeps a miner earning.

Bittensor Subnets

SSH Control Plane

Approval-Gated Ops

90s Monitor Pass

What the DevOps Engine Does

The DevOps Engine is the operations control plane between you and a bare GPU machine. It validates the box, reads the subnet's actual requirements from its official GitHub repository, installs exactly those requirements, launches the miner behind an approval gate, and then keeps watching the machine every ninety seconds for as long as the workload runs. Everything it does is recorded as resumable, auditable state: install steps persist to the database, so a half-finished install can be resumed where it stopped instead of restarted from scratch, and every step's full command output is kept for review.

For bring-your-own hosts the engine works over **SSH** — credentials are stored encrypted (AES-256-GCM), every command has a timeout, and the old scripted mock host was deliberately removed from the codebase: a machine either answers real commands or the engine reports the failure honestly. Rental deployments use a small **node daemon** instead: the daemon signs every telemetry POST and command poll with HMAC-SHA256, and it only ever **pulls** its own command queue — the platform never pushes into your machine. Five components carry the whole flow.

GUIDED ONBOARDING

Mining Journey

A five-step stepper — choose subnet, get a GPU host, validate, deploy & mine, connect & register. Progress is driven by real database and chain state, never by what you clicked.

HOST VALIDATION

Environment Inspector

A ten-step SSH pipeline that inventories the OS, GPU, driver, Docker, toolkit and CUDA, live-tests the GPU inside a container, and one-click fixes what is missing.

DEPLOYMENT

Requirement Installer

Builds a per-subnet install plan from the requirements profile scraped out of the subnet's own repository, then executes it step by step with auto, manual and approval gates.

TELEMETRY

Node Daemon

A tiny agent on rental machines streaming GPU utilization, temperature, memory and process state. Its secret is shown once, and every packet is HMAC-signed with replay protection.

ALWAYS-ON WATCH

DevOps Monitor

A ninety-second pass over every started deployment. It evaluates GPU health (process alive, thermals, utilization, daemon silence) and diffs the subnet against its previous state (hyperparameters, capacity, economics, hotkey churn). Findings land in the Trigger Center as deduplicated, approval-gated recommendations — recovery auto-resolves them when conditions return to normal.

What the engine deliberately never does

- ✗ It never creates or moves your wallet keys — coldkeys and hotkeys only ever live on your machine, and the wallet step is a manual gate that checks for their presence.
- ✗ It never registers your hotkey on the chain for you — registration must be signed by the coldkey that pays the burn.
- ✗ It never pushes commands into a host — the daemon pulls its own queue, and only actions you approved are ever enqueued.
- ✗ It never simulates a machine that is not real — mock hosts fail loudly instead of pretending to be live hardware.

CHAPTER 2

The Mining Journey — Five Steps

The DevOps view opens on a five-step journey. Both supported paths — a rented GPU with the daemon, or your own box reached over SSH — advance the same stepper, and a step only turns green when the underlying state is real. Here is what happens inside each step, and what you should personally verify before moving on.

1

Choose subnet

The engine resolves the subnet's on-chain GitHub repository and scrapes its README into a requirements profile: minimum VRAM, CUDA floor, Python or Docker runtime, infra services (Postgres, Redis, Kubernetes), ports and the miner endpoint.

YOU VERIFY The requirements dialog names the repo you expect, and the economics view shows incentive history worth your hardware.

2

Get a GPU host

Register a machine: host address, SSH port, user, and a password or private key (encrypted at rest) — or spin up a rental deployment, which provisions the box and enrolls its daemon.

YOU VERIFY The host card appears in the DevOps view with status **PENDING** and the address is reachable from where you are.

3

Validate

The inspector runs its ten-step pipeline over SSH (next chapter) and reports a verdict per step, with one-click fixes for everything it can safely install itself.

YOU VERIFY The host badge reads **READY**. A **NEEDS FIX** verdict is normal on fresh boxes — apply the fix, then re-validate.

4

Deploy & mine

The installer turns the requirements profile into a concrete plan — packages, runtime, repo clone, dependencies, environment file — and executes it step by step. Launching the miner is an explicit approval gate.

YOU VERIFY The install plan shows **deployed** with every step passed or skipped, and the verify step caught the service live with the axon port listening.

5

Connect & register

The wallet wizard renders the exact btcli commands for your wallet name, hotkey name and netuid, then verifies the hotkey against the live finney chain — returning your UID, incentive, consensus and immunity window.

YOU VERIFY A registered UID exists and the journey shows no "restart pending" hint. Register **last**: the hotkey should be registered only after the miner is already running.

CHAPTER 3

Host Validation — the Ten-Step Inspector

Before anything is installed, the inspector connects to the machine over SSH and walks ten checks, each persisted with its status, output, duration and remediation hint so the wizard is a resumable checklist rather than a one-shot log. Every check can re-run independently, and the fix flow marks a step **fixed** only after the re-check actually passes — never on trust.

#	Check	What it verifies	If it fails
1	Connect to GPU	SSH handshake to the host — the gate for everything else.	Check address, port, user, credentials, firewall/security group.
2	Detect OS	Distribution, version and architecture from /etc/os-release.	Use a supported Ubuntu LTS image; note the architecture.
3	Detect NVIDIA GPU	PCI probe plus nvidia-smi — model and total VRAM.	Install the NVIDIA driver for the card.
4	Validate Driver	Driver version and the CUDA capability it exposes.	Upgrade the driver; the fix flow offers the safe path.
5	Validate Docker	Docker engine and compose are installed and responding.	One-click install of docker-ce.
6	Validate Toolkit	nvidia-container-toolkit and the Docker nvidia runtime.	One-click toolkit install and runtime configuration.
7	Validate CUDA	Driver CUDA meets the platform minimum of 12.0.	Driver upgrade required — no shortcut.
8	Test GPU in Docker	docker run --gpus all launches a CUDA image and lists the GPU.	Runtime misconfiguration — apply the toolkit fix, re-test.
9	Install / Fix Missing	Remediation dispatcher for everything above (one-click, auto-safe).	Approve the fixes, then re-run the affected checks.
10	Configure Environment	Writes /root/.infranex/env with OS, GPU, driver and CUDA facts.	Re-run — the file feeds the deployment layer.

PENDING

VALIDATING

READY

NEEDS FIX

FAILED

Iron rule: do not deploy onto a host that is not **READY**. A needs-fix verdict is honest information, not an obstacle — the whole point of the pipeline is that a miner on the wrong host class never validates, and you find that out before burning compute, not after.

CHAPTER 4

The Install Plan — What Lands on Your Box

Once a host is ready and a subnet chosen, the installer compiles the requirements profile into a concrete plan and executes it one step at a time, persisting every step's full output. Three runtime paths exist: a **Python venv** for classic repositories, a **Docker image build** when the repo ships a Dockerfile, and an honest **manual provisioning gate** for cluster-shaped subnets whose own documentation requires the miner to run inside Kubernetes — the engine refuses to fake that with a one-click script.

#	Step	What happens	Gate
1	Match requirements to host	Compares host GPU, VRAM and driver CUDA against the subnet's floor (1 GB tolerance).	AUTO
2	OS packages	apt installs git, python3-venv, build-essential plus anything the repo hints.	AUTO
3	Runtime	Python venv under /opt/infranex/sn<netuid>, or Docker + NVIDIA container toolkit, or skipped for cluster subnets.	AUTO
4	Clone the subnet repo	Depth-1 clone of the schema-validated, on-chain linked GitHub URL.	AUTO
5	Dependencies	pip install -r requirements.txt, or uv sync for workspace repos, or docker build; documented services (Postgres, Redis) installed as system services.	AUTO
5b	Host prerequisites	Documented RAM-per-GPU sizing, storage and networking rules surfaced as real steps; Kubernetes subnets get a manual provisioning gate with README evidence.	MANUAL
6	Miner environment	Writes /opt/infranex/sn<netuid>/env — network, netuid, wallet name, hotkey name, GPU device.	AUTO
7	Wallet files present	Checks coldkey and hotkey exist under ~/.bittensor/wallets/... — the engine never creates or transfers keys.	MANUAL

#	Step	What happens	Gate
8	Launch the miner	systemd unit (auto-restart, logs to /var/log/infranex-miner-sn<netuid>.log) or a GPU container with the wallet directory mounted read-only.	APPROVAL
9	Verify	Service active, axon port listening, first log lines captured into the step output.	AUTO

AUTO

Executes directly and records the full output. Retryable any time; a failed command stops the step and attaches a remediation hint.

MANUAL

Belongs to you. Wallet files are confirmed by you — the engine only checks presence. Kubernetes subnets require you to provision the cluster with their official tooling first.

APPROVAL

The launch step. Nothing runs until you explicitly approve — this is the moment the machine starts burning compute.

Install states: running → waiting_approval → deployed, with failed and stopped as exits. A **safety stop** disables the service and parks the install; re-running the launch step from the wizard relaunches it without redoing the whole plan.

CHAPTER 5

The First 30 Minutes After the Miner Starts

The miner just launched — approval accepted, service started. This is the window where a five-second check now saves a five-hour debug later. Work top to bottom; the first three checks come straight out of the engine's verify step, and you can re-run all of them by hand over SSH at any time. Replace `<netuid>` with your subnet number and use the Docker variant only if the plan built a container.

☐ 1 — The service is actually live

The verify step already ran this, but confirm it yourself. Expect `active` (systemd) or `Up` (Docker).

```
systemctl is-active infranex-miner-sn<netuid>
# container path:
docker ps --filter name=infranex-miner-sn<netuid>
```

☐ 2 — The axon port is listening

The subnet's axon port (shown in the requirements profile) must accept connections — validators reach your miner here. No listener means the miner died right after start.

```
ss -tlnp | grep ':<axon-port> '
```

☐ 3 — The log tail looks sane

Read the first lines the verify step captured, then keep watching for a minute: wallet errors, port conflicts and chain-sync failures all show up here first.

```
tail -n 50 /var/log/infranex-miner-sn<netuid>.log
# container path:
docker logs --tail 50 infranex-miner-sn<netuid>
```

☐ 4 — The install plan reads deployed

Open the host card in the DevOps view: every step passed or skipped, none failed, overall state **deployed**. A failed step always carries a remediation hint — fix and retry that step, not the whole plan.

☐ 5 — Wallet keys are really on the host

You confirmed this at the manual gate, but if you overrode it, double-check now — a miner without keys is a service that logs errors forever.

```
ls ~/.bittensor/wallets/<wallet>/coldkey
ls ~/.bittensor/wallets/<wallet>/hotkeys/<hotkey>
```

☐ 6 — Register the hotkey — last, not first

With the miner running, follow the Connect & register wizard: it renders the exact btcli commands for your wallet, hotkey and netuid. The registration burn is signed by your coldkey — the engine never touches it.

```
btcli wallet register --netuid <netuid> \
  --wallet.name <wallet> --wallet.hotkey <hotkey> \
  --subtensor.network finney
```

☐ 7 — Verify the UID on the live chain

The wizard's verify hits live finney and returns your UID, incentive, consensus, emission and the immunity window with blocks remaining. Expect a UID immediately; incentive accrues over subsequent windows, so a zero there on day one is normal.

☐ 8 — Restart after registration if prompted

If the journey shows "restart pending", restart the miner so it comes up serving with the registered key. Re-run checks 1–3 after the restart.

☐ 9 — First monitor passes are clean

Within two to three minutes the DevOps view shows vitals for the deployment. Open the recommendations inbox and confirm no **CRITICAL** finding (process dead, thermal) was raised by the first passes.

The 30-minute rule: if anything failed at launch, the log tail in the verify step almost always names the cause — wrong wallet path, port conflict, or chain sync. Chapter 7 maps each failure to its fastest fix.

Ongoing Monitoring — What and When to Check

From the moment a deployment is started, an automatic monitor pass runs every ninety seconds and keeps roughly six hours of GPU samples per miner. You do not need to stare at graphs: the pass raises deduplicated findings into the recommendations inbox, and it auto-resolves them when conditions recover. Your job is to read the inbox and approve or reject each proposed action. The thresholds it uses are fixed and visible:

Signal	Condition	Severity	What it means
Daemon missing	No daemon enrolled for the deployment	INFO	Install the daemon to get telemetry.
Daemon silent	No telemetry for 10 minutes	WARNING	Host network or daemon process issue.
Miner process dead	Dead for 2 consecutive passes	CRITICAL	Restart action offered — approve it.
GPU temperature	85 °C on 2 passes	CRITICAL	Thermal throttling imminent — cooling, airflow, ambient.
GPU temperature	78 °C on 2 passes	WARNING	Headroom shrinking; check dust and fans.
GPU idle	Utilization under 30% on 3 passes while process alive	WARNING	Miner is up but not working — subnet demand or config.
Subnet drift	Hyperparameters changed (tempo, immunity, max UIDs)	EVENT	Review the subnet's new rules.
Capacity move	Registered UID count shifted	EVENT	Competition for slots changed.
Economics shift	Median rewarded incentive moved $\pm 50\%$	EVENT	Re-check the ROI of this subnet.
Hotkey UID moved	Your UID changed or vanished	WARNING	Deregistration churn — re-register.

A checking cadence that works

When	What you check	Where
Every 90 s (automatic)	Process, thermals, utilization, daemon silence, subnet drift. ~6 h of GPU samples retained.	Nothing — the engine watches.
Every login	Vitals row per miner; the recommendations inbox — approve or reject each proposed action.	DevOps view cockpit.
Daily	Chain side: UID active, incentive accruing, last-update age small, immunity blocks remaining.	Wallet wizard verify / chain.
Weekly	Drift events over the week, benchmark harness results, alert channels still configured and reachable.	Strategy board, Autopilot panel.

Approval gating: findings propose, you dispose. A critical process-death event offers a daemon restart, but nothing executes until you approve it in the Trigger Center. Recovery auto-resolves findings, so the inbox stays a worklist — not a landfill.

CHAPTER 7

Troubleshooting Quick Reference

Failed steps do not leave you guessing: each one carries the engine's own remediation hint in its output. The table below collects the hints the engine attaches, plus the two or three causes that dominate each case in practice. Find the failing step title in the install wizard, apply the fix, and re-run that single step — passed steps are never redone.

Failing step	Typical cause	What to do
Install OS packages	apt sources or network broken on the host	Fix distro sources / connectivity, retry the step.
Install Docker / toolkit	curl or package network unavailable	Install docker-ce + nvidia-container-toolkit manually, retry.
Build Docker image	Failing Dockerfile layer (deps, base pull)	Read the layer in the step output, fix, retry.
Create Python venv	Repo pins a specific Python version	Install it (deadsnakes PPA) or adjust the plan.
Clone the repo	Repo private or no outbound network	Confirm the repo is public and reachable from the host.
Python dependencies	Heavy CUDA wheels, low RAM or disk	Free resources, then retry the step.

Failing step	Typical cause	What to do
Wallet files present	Keys never copied to the host	scp ~/.bittensor/wallets/... onto the host, then confirm.
Launch the miner	Wallet path, port conflict, chain errors	Read the verify step's log tail — it names the cause.
Verify the miner	Service not active, axon not listening	journalctl -u infranex-miner-sn<netuid> — wallet, ports, sync.

Safety stop and relaunch: the stop action disables the systemd unit or removes the container and marks the install stopped. Nothing is torn down — re-running the launch step starts the miner again with the same environment, which is exactly what you want after maintenance or a registration restart.

Engine self-check: the DevOps Engine watches itself too. The System Health view reports the state of the live chain connection, the price oracles, GitHub scraping and the monitor pass cadence, and the audit log records every approval and every step execution. If monitor passes stop advancing or a view reports stale data, fix the platform first — the miner is usually the last thing that is wrong.

Six things to remember

01 Validate before you deploy

Nothing runs on a host that is not READY. Needs-fix is information — fix, re-validate, proceed.

02 Two gates belong to you

Wallet files are a manual gate; the launch is an approval gate. The engine never crosses either one for you.

03 Register last

The hotkey is registered only after the miner is running — then verify the UID against the live chain.

04 First 30 minutes are a checklist

Service live, axon listening, log tail sane, plan deployed, keys present, monitor passes clean.

05 Read the inbox, not the graphs

The 90-second pass watches thermals, utilization, process state and drift — you judge its recommendations.

06 Watch the chain weekly

Incentive, last-update age and immunity remaining tell you whether the miner is actually earning.

The engine runs the operations.
The judgment stays with you.